Corso di Fondamenti di Informatica 2
CdL Ingegneria Informatica
Ing. Franco Zambonelli

ARCHITETTURA DEGLI ELABORATORI: MICROPROCESSORI INTEL 8086/8088

Lucidi Realizzati in Collaborazione con:

Prof. Letizia Leonardi
Università di Modena

Prof. Antonio Corradi
Università di Bologna

ARRAY MONODIMENSIONALI

char va[NA]**;**
sizeof(va) = NA * sizeof(char) = NA byte

short int vb[NB];
sizeof(vb) = NB * sizeof(short int) = 2 * NB byte

DataType vc[NC];
sizeof(vc) = NC * sizeof(DataType) = 6 * NC byte

DEFINIZIONE DI VETTORI GLOBALI

```
va      DB  NA dup (?)
vb      DW  NB dup (?)              oppure
vb      DB  (2 * NB) dup (?)
vc      DB  (6 * NC) dup (?)
```

DEFINIZIONE DI VETTORI LOCALI

```
sub  sp, NA          ; va – se NA pari
sub  sp, NA + 1      ; va – se NA dispari
sub  sp, 2 * NB      ; vb
sub  sp, 6 * NC      ; vc
```

Il valore finale di SP è l'indirizzo di partenza del vettore
E' però necessario determinare l'**indirizzo di partenza del vettore in funzione di BP** e non di SP

ACCESSO AL GENERICO ELEMENTO V[k]

&V[k] = &V[0] + **k** * sizeof(V[0])

L'indirizzo deve essere calcolato **in fase di esecuzione**
- il valore di k varia di volta in volta

Si supponga che:
- l'indice k sia memorizzato in SI
- &v[k] debba essere memorizzato in BX

ACCESSO A VETTORI GLOBALI

```
LEA  BX, [va + SI]

SHL  SI, 1
LEA  BX, [vb + SI]
SHR  SI, 1

MOV  AX, SI
MOV  DX, 6
IMUL  DX              ;  DX:AX <- 6 * k
MOV  BX, AX
LEA  BX, [vc + BX]
```

ACCESSO A VETTORI LOCALI

Sostituire va/vb/vc con: BP – xxx

ARRAY BIDIMENSIONALI

char ma[RA][CA]**;**
  sizeof(ma) = RA * CA byte

short int mb[RB][CB];
  sizeof(mb) = RB * CB * 2 byte

DataType mc[RC][CC];
  sizeof(vc) = RC * CC * 6 byte

DEFINIZIONE DI MATRICI GLOBALI

```
ma    DB RA * CA dup (?)
mb    DW RB * CB dup (?)          oppure
mb    DB  (2 * RB * CB) dup (?)
mc    DB  (6 * RC * CC) dup (?)
```

DEFINIZIONE DI MATRICI LOCALI

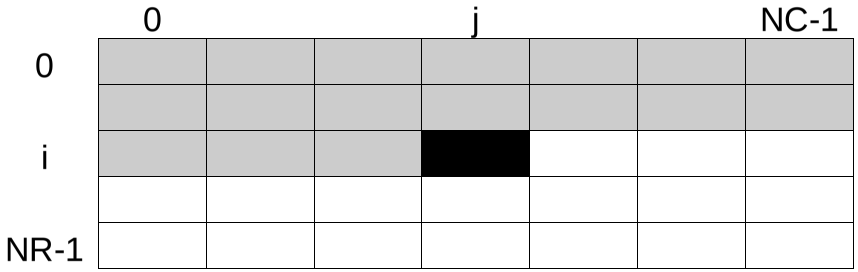
```
sub  sp, RA * CA        ; ma – se RA*CA pari
sub  sp, RA * CA + 1    ; ma – se RA*CA dispari
sub  sp, 2 * RB * CB    ; mb
sub  sp, 6 * RC * CC    ; mc
```

Il valore finale di SP è l'indirizzo di partenza del vettore
E' però necessario determinare l'**indirizzo di partenza del vettore in funzione di BP** e non di SP

ACCESSO AL GENERICO ELEMENTO M[i][j]

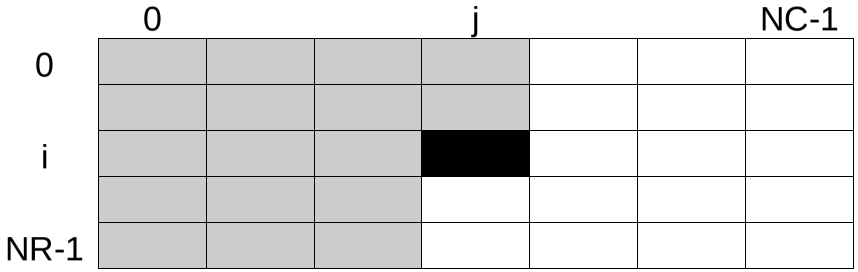
- **Memorizzazione per righe** (convenzione C):

&M[i][j] = &M[0][0] + (i * NC + j) * sizeof(M[0][0])



- **Memorizzazione per colonne**:

&M[i][j] = &M[0][0] + (i + j * NR) * sizeof(M[0][0])



L'indirizzo deve essere calcolato **in fase di esecuzione**
- i valori di i e j variano di volta in volta

Esempi:

```
int vector[] = {1,2,3,4,5,6,7,8,9,0};

int matrix[4][4] = {{1,0,0,0},{0,1,0,0},{0,0,1,0},{0,0,0,1}};

typedef struct
        {
        int giorno,mese,anno;
        } DataType;

DataType d1,d2[5],d3[2][3],*d4,*d5 = &d1,*d6 = &d2[3];

char *str = "Fondamenti di informatica II";
```

```
_DATA   segment word public 'DATA'
vector  dw      1,2,3,4,5,6,7,8,9,0
matrix  dw      1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1
                ; oppure  3 dup (1,4 dup (0)),1
d1      dw      3 dup (0)
d2      dw      15 dup (0)
d3      dw      18 dup (0)
d4      dw      0
d5      dw      d1
d6      dw      d2+18
str     dw      s@
s@      db      'Fondamenti di informatica II',0
_DATA   ends
```

```
        . . .
        vector[0] = 0;
        vector[1] = 1;
        d4 = &d1;
        d1 = *d6;
        . . .


_TEXT   segment byte public 'CODE'
        assume  cs:_TEXT,ds:_DATA
        . . .
        mov     word ptr [vector],0       ; vector[0] = 0
        mov     word ptr [vector+2],1     ; vector[1] = 1
        mov     word ptr d4,offset d1     ; d4 = &d1
;
;       d1 = *d6;
;
        mov     ax,offset d1
        push    ds
        push    ax
        push    ds
        push    word ptr d6
        mov     cx,6
        call    N_SCOPY@
; elimina automaticamente gli argomenti dallo stack
        . . .
_TEXT   ends
```


Esempio 1

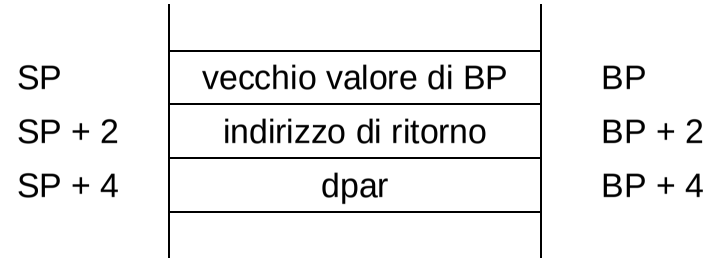
```
typedef struct
        {
        int giorno,mese,anno;
        } DataType;

static DataType dtemp;

void fun(DataType *dpar)
        {
        dtemp.giorno = dpar->giorno;
        dtemp.mese = dpar->mese;
        dtemp.anno = dpar->anno;
        }
```

```
_DATA   segment word public 'DATA'
dtemp   dw      3 dup (?)
_DATA   ends

_TEXT   segment byte public 'CODE'
        assume cs:_TEXT,ds:_DATA
_fun    proc    near
        push    bp
        mov     bp,sp
```




```
        mov     bx,word ptr [bp+4]   ; bx <- dpar
;               dtemp.giorno = dpar->giorno;
        mov     ax,word ptr [bx]
        mov     word ptr dtemp,ax
;               dtemp.mese = dpar->mese;
        mov     ax,word ptr [bx+2]
        mov     word ptr dtemp+2,ax
;               dtemp.anno = dpar->anno;
        mov     ax,word ptr [bx+4]
        mov     word ptr dtemp+4,ax
        pop     bp
        ret
_fun    endp
_TEXT   ends
        end
```


Esempio 2

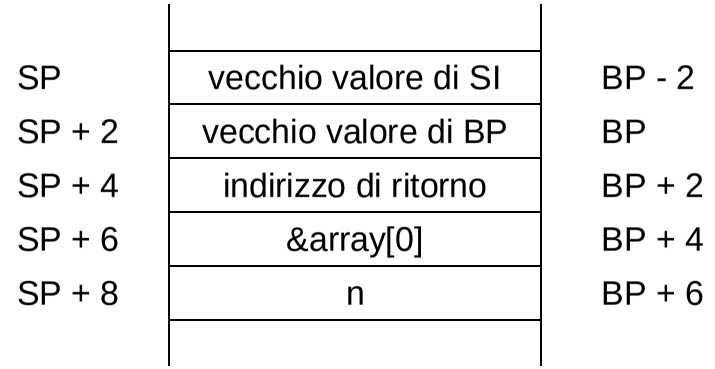
```
static int stArray[15];

void fun(int array[ ],int n)
        {
        int j;

        for (j = 0; j < n; j++) stArray[j] = array[j];
        }
```

```
_DATA   segment word public 'DATA'
stArray dw      15 dup (?)
_DATA   ends

_TEXT   segment byte public 'CODE'
        assume  cs:_TEXT,ds:_DATA
_fun    proc    near
        push    bp
        mov     bp,sp
        push    si
```




```
        mov   bx,word ptr [bp+4]   ; bx <- &array[0]
  ;             for (j = 0; j < n; j++)
        mov   si,0            ; j = 0;
cicla:  cmp   si,word ptr [bp+6]
        jge   fine            ; if(j >= n) goto fine;
  ;             stArray[j] = array[j];
        shl   si,1            ; j*sizeof(int)
        mov   ax,word ptr [bx+si]   ; ax <- array[j]
        mov   word ptr [stArray+si],ax
        shr   si,1
        inc   si              ; j++
        jmp   cicla
fine:   pop   si
        pop   bp
        ret
_fun    endp
_TEXT   ends
        end
```


Esempio 3

```
void upperText(char *string)
        {
        for (; *string; string++)
          if('a' <= *string && *string <= 'z')
            *string &= 0xDF;
        }

void lowerText(char *string)
        {
        for (; *string; string++)
          if('A' <= *string && *string <= 'Z')
            *string |= 0x20;
        }
```

Codici ASCII:

A = 65 = 0x41 a = 97 = 0x61
Z = 90 = 0x5Az = 122 = 0x7A

```
        public  _upperText
        public  _lowerText
_TEXT   segment byte public 'CODE'
        assume  cs:_TEXT
```

```
  ;
  ;       void upperText(char *string)
  ;
_upperText      proc  near
          push    bp
          mov     bp,sp
          mov     bx,word ptr [bp+4]   ;  bx <- string
cicla1:
          cmp     byte ptr [bx], 0
          je      fine1         ;  if(*string == 0) goto fine;

  ;         if(*string < 'a' || *string > 'z') goto continua1;

          cmp     byte ptr [bx], 'a'
          jl      continua1
          cmp     byte ptr [bx], 'z'
          jg      continua1

  ;         *string &= 0xDF;

          and     byte ptr [bx], 0DFh
continua1:
          inc     bx                    ;  string++;
          jmp     cicla1
fine1:
          pop     bp
          ret
_upperText      endp
```



```
  ;
  ;       void lowerText(char *string)
  ;
_lowerText      proc  near
          push    bp
          mov     bp,sp
          mov     bx,word ptr [bp+4]   ;  bx <- string
cicla2:
          cmp     byte ptr [bx], 0
          je      fine2         ;  if(*string == 0) goto fine;

  ;         if(*string < 'A' || *string > 'Z') goto continua2;

          cmp     byte ptr [bx], 'A'
          jl      continua2
          cmp     byte ptr [bx], 'Z'
          jg      continua2

  ;         *string |= 0x20;

          or      byte ptr [bx], 020h
continua2:
          inc     bx                    ;  string++;
          jmp     cicla2
fine2:
          pop     bp
          ret
_lowerText      endp
_TEXT   ends
          end
```

Esempio 4

```
void elabWord(char *string,int);

void findWords(char *string)
        {
        char *p1,*p2;

        for (p1 = string;;)
          {
          while (*p1 == ' ') p1++;
          if(*p1 == '\0') break;
          p2 = p1;
          while (*p1 && *p1 != ' ') p1++;
          elabWord(p2,p1-p2);
          }
        }
```

```
          public  _findWords
          extrn   _elabWord:near
_TEXT   segment byte public 'CODE'
          assume      cs:_TEXT
p1        equ     si
p2        equ     di
```



```
  ;
  ;       void findWords(char *string)
  ;
_findWords      proc  near
          push    bp
          mov     bp,sp
          push    si              ;  utilizzato per p1
          push    di              ;  utilizzato per p2

  ;               for (p1 = string;;)

          mov     p1,word ptr [bp+4]
ciclofor:

  ;                 while (*p1 == ' ') p1++;

while1:
          cmp     byte ptr [p1], ' '
          jne     exit1
          inc     p1
          jmp     while1
exit1:

  ;                 if(*p1 == '\0') break;

          cmp     byte ptr [p1],0
          je      fine

          mov     p2, p1                  ;  p2 = p1;
```

```
   ;                    while (*p1 && *p1 != ' ') p1++;
while2:
            cmp     byte ptr [p1], 0
            je      exit2
            cmp     byte ptr [p1], ' '
            je      exit2
            inc     p1
            jmp     while2
exit2:
   ;                    elabWord(p2,p1-p2);
            mov     ax,p1
            sub     ax,p2;  ax <- p1 - p2
            push    ax
            push    p2
            call    near ptr _elabWord
            add     sp, 4           ;  4 clock, 4 byte
;    2  x   pop reg                 ;  2 x (8 clock, 1 byte)
            jmp     ciclofor
fine:
            pop     di
            pop     si
            pop     bp
            ret
_findWords  endp
_TEXT   ends
            end
```


Esempio 6

```
fatt(int n)
        {
        if(n <= 1)  return 1;
        else  return n*fatt(n-1);
        }
```

```
            public  _fatt
_TEXT   segment byte public 'CODE'
            assume  cs:_TEXT
_fatt       proc    near
            push    bp
            mov     bp,sp
```

```
        mov     bx,word ptr [bp+4]          ;  bx <- n

  ;             if(n <= 1)  return 1;

        cmp     bx,1
        jg      else            ;  if(n > 1) goto else;
        mov     ax,1
        jmp     ritorno

else:

  ;             else  return n*fatt(n-1);

        mov     ax,bx
        dec     ax                      ;  ax <- n - 1
        push    ax
        call    near ptr _fatt
        add     sp, 2                   ;  pop cx
        imul    word ptr [bp+4]         ;  il risultato e' in ax

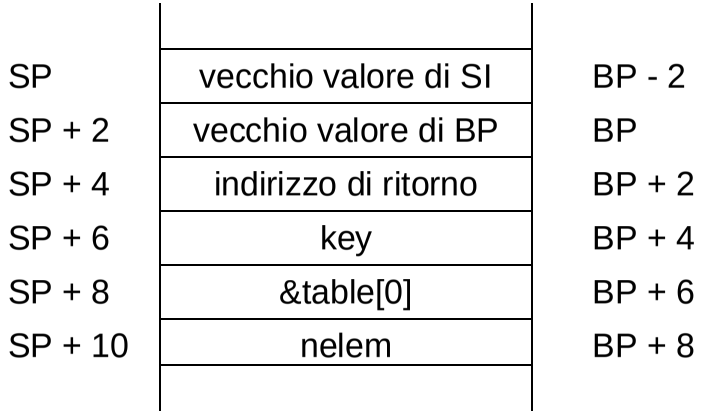
ritorno:
        pop     bp
        ret
_fatt   endp
_TEXT   ends
        end
```


Esempio 7

```
find(int key,int table[ ],int nelem)
        {
        int k,k1,k2;

        for (k1 = 0, k2 = nelem - 1; k1 <= k2; )
          {
          k = (k1 + k2) / 2;
          if(key < table[k]) k2 = k - 1;
          else if(key > table[k]) k1 = k + 1;
          else return k;
          }
        return -1;
        }
```

```
        public  _find
_TEXT   segment byte public 'CODE'
        assume  cs:_TEXT
_find   proc    near
        push    bp
        mov     bp,sp
        push    si
k       equ     ax
k1      equ     cx
k2      equ     dx
```




```
key     equ     word ptr [bp+4]
nelem   equ     word ptr [bp+8]

        mov     bx, word ptr [bp+6]  ;  bx <- &table[0]

;               for (k1 = 0, k2 = nelem - 1; k1 <= k2; )

        mov     k1, 0
        mov     k2, nelem
        dec     k2
cicla:
        cmp     k1, k2
        jg      fine            ;  if(k1 > k2) goto fine;

;                 k = (k1 + k2) / 2;

        mov     k, k1
        add     k, k2
        sar     k, 1
```



```
;                 if(key < table[k]) k2 = k - 1;

        mov     si, k
        sal     si, 1           ;  si <- k*sizeof(int)
        mov     si,[bx+si]      ;  si <- table[k]
        cmp     si,key
        jle     step2   ; if(table[k] <= key) goto step2;
        mov     k2, k
        dec     k2
        jmp     cicla

;                 else if(key > table[k]) k1 = k + 1;

step2:  jge     step3   ; if(table[k] >= key) goto step3;
        mov     k1, k
        inc     k1
        jmp     cicla

;                 else return k;

step3:
;       mov     ax, k -- ax coincide con k --
        jmp     ritorno

fine:   mov     ax, -1

ritorno: pop    si
        pop     bp
        ret
_find   endp
_TEXT   ends
        end
```

Esempio 8

```
print_string macro string
        mov     ah,9
        lea     dx,string
        int     21h
        endm

exit    macro
        mov     ah,4ch
        mov     al, 0
        int     21h
        endm

load_ds macro segName
        mov     ax,segName
        mov     ds,ax
        assume  ds:segName
        endm

_STACK segment word STACK 'STACK'
        db      32000 dup (?)
_STACK ends

_DATA   segment word public 'DATA'
Size    equ     6
string  db      Size dup (?),'$'
op1     dw      -4490           ; un numero qualsiasi
op2     dw      -80             ; un numero qualsiasi
_DATA   ends
```



```
_TEXT   segment byte public 'CODE'
        assume  cs:_TEXT,ss:_STACK
main    proc
        load_ds  _DATA
        mov     ax, op1
        add     ax, op2
        lea     bx, string
        mov     cx, Size
        call    cvt_it
        print_string string
        exit
main    endp

cvt_it  proc
;       in ingresso:
;         ax = numero con segno da convertire
;         bx  = offset della stringa destinazione
;         cx = lunghezza della stringa destinazione
;       in uscita:
;         ax, bx, cx e dx sono privi di significato
        push    si
        push    di
        mov     si,10
        mov     di,cx           ;  lunghezza dest.
        mov     cl,'+'
        test    ax,ax
        jns     cicla           ;  se ax>=0 salta a cicla
        neg     ax
        mov     cl,'-'
```

```
cicla:
        mov     dx,0
        div     si              ;  dx:ax  /  si
                                ;  ax <- quoziente, dx <- resto
        add     dl,'0'          ;  trasformo in ASCII il resto
        dec     di
        mov     byte ptr [bx+di],dl
        cmp     ax,0
        jne     cicla           ;  ripeti se a<>0
fill:
        dec     di
        mov     byte ptr [bx+di],cl     ;  segno o bianco
        mov     cl,' '
        cmp     di,0
        jne     fill
        pop     di
        pop     si
        ret
cvt_it  endp

_TEXT   ends
        end     main
```


Esempio 9

```
read_char macro car
        mov     ah,1
        int     21h
        mov     car,al
        endm

print_char macro car
        mov     ah,2
        mov     dl,car
        int     21h
        endm

LF      equ     0ah
CR      equ     0dh
FALSE   equ     0
TRUE    equ     1

_data   segment     'DATA'
Size    =       20
string  db      Size dup (?)
_data   ends
```

```
_TEXT   segment 'CODE'
        assume  cs:_TEXT,ss:_stack
main    proc
        load_ds     _data
        mov     cx,Size
        mov     di,0
ciclo1:
        read_char [string+di]
        cmp     byte ptr [string+di],CR
        je      step2
        inc     di
        loop    ciclo1
step2:
        print_char LF
        print_char CR
        lea     bx,string
        mov     cx,di
        call    bubble

        mov     cx,di
        mov     di,0
ciclo2:
        print_char [string+di]
        inc     di
        loop    ciclo2
        exit
main    endp
```



```
bubble  proc

;       in ingresso:
;         bx = offset array di caratteri da ordinare
;         cx = numero di caratteri
;       in uscita:
;         ax, bx, cx e dx sono privi di significato

        push    si
        dec     cx
        mov     dx,cx ; dx <- nelem-1
cicl1:  mov     ah,FALSE ; nessuno scambio
        mov     cx,dx ; cx <- nelem-1
        mov     si,bx ; si punta al primo elemento
cicl2:  mov     al,byte ptr [si]
        cmp     al,byte ptr [si+1]
        jle     vai
        xchg    byte ptr [si+1],al
        mov     byte ptr [si],al
        mov     ah,TRUE
vai:    inc     si
        loop    cicl2
        cmp     ah,TRUE
        je      cicl1
        pop     si
        ret
bubble  endp

_TEXT   ends
        end     main
```

Esempio 10

```
_data   segment     'DATA'
Size    =       20
string  db      Size dup (?),0
_data   ends

_TEXT   segment 'CODE'
        assume  cs:_TEXT,ss:_stack
main    proc
        load_ds     _data

        mov     ax,offset string
        push    ax
        mov     ax,Size
        push    ax
        call    gets
        add     sp, 4
        mov     di,ax           ;  lunghezza stringa

        print_char LF
        print_char CR

        mov     ax,offset string
        push    ax
        push    di
        call    bubble
        add     sp, 4
```



```
        mov     ax,offset string
        push    ax
        call    leftTrim
        add     sp, 2

        mov     ax,offset string
        push    ax
        call    puts
        add     sp, 2

        exit
main    endp
```

```
gets    proc    ; int gets(int size,char *string)

;       in ingresso:
;         [bp+4] = numero massimo di caratteri
;         [bp+6] = offset della stringa destinazione
;       in uscita:
;         ax contiene il numero di caratteri letti

        push    bp
        mov     bp,sp
        mov     cx,[bp+4]
        mov     bx,[bp+6]
gets1:
        mov     ah,1
        int     21h
        cmp     al,CR
        je      gets2
        mov     byte ptr [bx],al
        inc     bx
        loop    gets1
gets2:
        mov     byte ptr [bx],0         ; terminatore C
        mov     ax,bx
        sub     ax,[bp+6]
        pop     bp
        ret
gets    endp
```



```
puts    proc    ; void puts(char *string)

;       in ingresso:
;         [bp+4] = offset della stringa destinazione

        push    bp
        mov     bp,sp
        mov     bx,[bp+4]
        mov     ah,2
puts1:
        mov     dl,byte ptr [bx]
        cmp     dl,0
        je      puts2
        int     21h
        inc     bx
        jmp     puts1
puts2:
        pop     bp
        ret
puts    endp
```

```
leftTrim proc     ; void leftTrim(char *string)

;           in ingresso:
;             [bp+4] = offset della stringa destinazione

            push   bp
            mov    bp,sp
            push   si
            mov    bx,[bp+4]
            mov    cx,0           ;  numero di bianchi
left1:
            cmp    byte ptr [bx],' '
            jne    left2
            inc    cx
            inc    bx
            jmp    left1
left2:
            cmp    cx,0
            je     left5
            mov    si,bx          ;  offset sorgente

            mov    cx,1           ;  caratteri da spostare
left3:
            cmp    byte ptr [bx],0
            je     left4
            inc    cx
            inc    bx
            jmp    left3
```



```
left4:
            push   di
            mov    di,[bp+4]      ;  offset destinazione
            mov    ax,ds
            mov    es,ax
            cld
rep         movsb
            pop    di
left5:
            pop    si
            pop    bp
            ret
leftTrim    endp


bubble      proc   ; void bubble(int size,char *string)

;           in ingresso:
;             [bp+4] = numero di caratteri
;             [bp+6] = offset dell'array di caratteri

            push   bp
            mov    bp,sp
            push   si
```

```
        mov     bx,[bp+6]     ;  offset array di caratteri
        mov     dx,[bp+4]
        dec     dx          ; dx <- numero di caratteri - 1
bubble1:
        mov     ah,FALSE ; nessuno scambio
        mov     cx,dx       ; cx <- numero di caratteri - 1
        mov     si,bx       ; si punta al primo elemento
bubble2:
        mov     al,byte ptr [si]
        cmp     al,byte ptr [si+1]
        jle     bubble3
        xchg    byte ptr [si+1],al
        mov     byte ptr [si],al
        mov     ah,TRUE
bubble3:
        inc     si
        loop    bubble2
        cmp     ah,TRUE
        je      bubble1
        pop     si
        pop     bp
        ret
bubble  endp

_TEXT   ends
        end     main
```